

DelhiHousePricePrediction

The aim of this project to predict the price of the houses in Delhi, in various localities, based on the data present in the dataset. The dataset is from Kaggle. The project aims to predict the house price, by analysing the features such as area, number of bedrooms, locality and many more. The dataset has 1259 rows and 11 columns.

Data Dictionary

ColumnName	Description
Area	Area of the house in square feet
BHK	Number of bedrooms
Bathroom	Number of bathrooms
Furnishing	Furnishing status
Locality	Locality of the house
Parking	Number of parking available
Price	Price of the house in INR
Status	property's status as in 'ready to move' or still under construction
Transaction	It's a new property or being re-sold
Type	Type of the property
Per_Sqft	Price per square feet

```
In []: #Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns

In []: #Loading the dataset
df = pd.read_csv('MagicBricks.csv')
df.head()
```

Out[]:

Area	BHK	Bathroom	Furnishing	Locality	Parking	Price	Status
0800.0	3	2.0	Semi-Furnished	Rohini Sector25	1.0	6500000	Ready_to_move
1750.0	2	2.0	Semi-Furnished	JR Designers Floors, Rohini Sector24	1.0	5000000	Ready_to_move
2950.0	2	2.0	Furnished	Apartment, Rohini Sector13	1.0	15500000	Ready_to_move
3600.0	2	2.0	Semi-Furnished	Rohini Sector24	1.0	4200000	Ready_to_move
4650.0	2	2.0	Semi-Furnished	Rohini Sector24 carpet area 650 sqft statusR...	1.0	6200000	Ready_to_move

DataPreprocessing1

In[]:

```
#Checkingtheshapeofthedataset
df.shape
```

Out[]:(1259, 11) In

[]:

```
#Checkingfornull/missingvalues
df.isnull().sum()
```

Out[]:

Area	0
BHK	0
Bathroom	2
Furnishing	5
Locality	0
Parking	33
Price	0
Status	0
Transaction	0
Type	5
Per_Sqft	241

dtype:int64

The dataset has five columns with missing values - Parking, Bathroom, Furnishing, Type andPer_Sqft.FindingvalueforPer_Sqftisquiteeasy.We havetodividePricebyAreato getPer_Sqft.TofindthemiissingvaluesinParking,Bathroom,FurnishingandType,Iwill replace the missing values with the mode of them.

In[]:

```
#ReplacingmissingvalueinPer_Sqft
df['Per_Sqft']=df['Per_Sqft'].fillna(df['Price']/df['Area'])
```

```
In[]: #ReplacingmissingvaluesinParking,Bathroom,FurnishingandType
df['Parking'].fillna(df['Parking'].mode()[0],inplace=True)
df['Bathroom'].fillna(df['Bathroom'].mode()[0],inplace=True)
df['Furnishing'].fillna(df['Furnishing'].mode()[0], inplace=True)
df['Type'].fillna(df['Type'].mode()[0], inplace=True)
```

```
In[]: #Checkingformissingvalues
df.isnull().sum()
```

```
Out[]: Area          0
BHK              0
Bathroom         0
Furnishing       0
Locality         0
Parking          0
Price            0
Status           0
Transaction      0
Type             0
Per_Sqft         0
dtype:int64
```

Checking datatype of each column

```
In[]: df.dtypes
```

```
Out[]: Area          float64
BHK              int64
Bathroom         float64
Furnishing       object
Locality         object
Parking          float64
Price            int64
Status           object
Transaction      object
Type             object
Per_Sqft         float64
dtype: object
```

ParkingandNumber ofbathrooms,cam't bein floatso,converting theminto integer

```
In[]: #Typecasting
df[['Parking','Bathroom']].astype('int64')
```

Out[]:

	Parking	Bathroom
0	1	2
1	1	2
2	1	2
3	1	2
4	1	2
...	...	...
1254	3	5
1255	3	2
1256	3	3
1257	1	2
1258	1	3

1259 rows × 2 columns

Unique value count in each column

In[]:

```
#Uniquevaluecount
df.nunique()
```

Out[]:

Area	315
BHK	8
Bathroom	7
Furnishing	3
Locality	365
Parking	9
Price	284
Status	2
Transaction	2
Type	2
Per_Sqft	433
dtype:int64	

Valuecountforeachvalueineach variable

In[]:

```
#Valuecountforeachvalue
print(df['Area'].value_counts(),'\n',df['BHK'].value_counts(),'\n',df['Bathroo
```

```
Area
900.0      67
1500.0     50
1800.0     48
1000.0     42
1600.0     38
..
150.0      1
3250.0     1
4000.0     1
5500.0     1
11050.0    1
Name: count, Length: 315, dtype: int64
BHK
3          541
2          367
4          220
1           96
5           27
6            6
7            1
10           1
Name: count, dtype: int64
Bathroom
2.0        553
3.0        355
1.0        146
4.0        138
5.0         57
6.0          7
7.0           3
Name: count, dtype: int64
Parking
1.0        829
2.0        341
3.0         54
4.0         16
5.0          7
39.0         7
114.0         3
9.0           1
10.0           1
Name: count, dtype: int64
Furnishing
Semi-Furnished    713
Unfurnished       363
Furnished         183
Name: count, dtype: int64
Status
Ready_to_move    1184
Almost_ready      75
Name: count, dtype: int64
Transaction
Resale           781
New_Property     478
Name: count, dtype: int64
Type
Builder_Floor    666
Apartment        593
Name: count, dtype: int64
```

```

Per_Sqft
12500.000000    37
3524.000000     28
6667.000000     27
14818.000000    24
6154.000000     18
..
11826.086957     1
11391.304348     1
9066.666667      1
6701.030928      1
16333.000000      1
Name:count, Length:433, dtype:int64

```

Adding new area column in SqYards for better understanding of area

```
In[]: df['Area_Yards'] = df['Area']/9
```

Grouping the House Locality

```
In[]: #Unique Localities
df['Locality'].unique()
```

Since there are so many localities in the dataset, I have decided to take only top 10 localities and list the remaining localities as 'other' in the dataset. It will help in analysing the locality of the house in a better way.

```
In[]: def grp_local(locality):
    locality = locality.lower() # avoid case sensitive
    if 'rohini' in locality:
        return 'Rohini Sector'
    elif 'dwarka' in locality:
        return 'Dwarka Sector'
    elif 'shahdara' in locality:
        return 'Shahdara'
    elif 'vasant' in locality:
        return 'VasantKunj'
    elif 'paschim' in locality:
        return 'PaschimVihar'
    elif 'alaknanda' in locality:
        return 'Alaknanda'
    elif 'vasundhar' in locality:
        return 'VasundharaEnclave'
    elif 'punjabi' in locality:
        return 'PunjabiBagh'
    elif 'kalkaji' in locality:
        return 'Kalkaji'
    elif 'lajpat' in locality:
        return 'LajpatNagar'
    else:
        return 'Other'
```

```
In[]: df['Locality'].value_counts()
```

```
Out[: Locality
      Other 716
      LajpatNagar 90
      DwarkaSector 87
      RohiniSector 75
      Shahdara 75
      Alaknanda 58
      VasantKunj 35
      Kalkaji 32
      PunjabiBagh 31
      PaschimVihar 30
      VasundharaEnclave 30
      Name:count,dtype:int64
```

```
In[: #UsingZ-scoretoremoveoutliers
      fromscipyimportstats
      #Zscore
      z=np.abs(stats.zscore(df[df.dtypes[df.dtypes!='object'].index]))
      #Removingoutliers
      df=df[(z<3).all(axis=1)]
```

Descriptive Statistics

```
In[ ]: #Checkingdescriptivesatisticsofthedata
      df.describe()
```

Out[]:

	Area	BHK	Bathroom	Parking	Price	Per_Sqft
count	1189.000000	1189.000000	1189.000000	1189.000000	1.189000e+03	1189.000000
mean	1296.421567	2.735913	2.483600	1.410429	1.852459e+07	12629.785274
std	750.284776	0.859232	0.952107	0.719913	1.772598e+07	8434.085021
min	28.000000	1.000000	1.000000	1.000000	1.000000e+06	1250.000000
25%	800.000000	2.000000	2.000000	1.000000	5.510000e+06	6526.000000
50%	1150.000000	3.000000	2.000000	1.000000	1.350000e+07	10943.000000
75%	1600.000000	3.000000	3.000000	2.000000	2.490000e+07	16584.000000
max	5220.000000	5.000000	5.000000	10.000000	9.300000e+07	72000.000000



```
In[: df.head(10)
```

Out[]:

	Area	BHK	Bathroom	Furnishing	Locality	Parking	Price	Status	
0	800.0	3	2.0	Semi-Furnished	Rohini Sector	1.0	6500000	Ready_to_move	N
1	750.0	2	2.0		Rohini Sector	1.0	5000000	Ready_to_move	N
2	950.0	2	2.0	Furnished	Rohini Sector	1.0	15500000	Ready_to_move	
3	600.0	2	2.0		Rohini Sector	1.0	4200000	Ready_to_move	
4	650.0	2	2.0	Semi-Furnished	Rohini Sector	1.0	6200000	Ready_to_move	N
5	1300.0	4	3.0		Rohini Sector	1.0	15500000	Ready_to_move	N
6	1350.0	4	3.0	Semi-Furnished	Rohini Sector	1.0	10000000	Ready_to_move	
7	650.0	2	2.0		Rohini Sector	1.0	4000000	Ready_to_move	N
8	985.0	3	3.0	Unfurnished	Rohini Sector	1.0	6800000	Almost_ready	N
9	1300.0	4	4.0		Rohini Sector	1.0	15000000	Ready_to_move	N

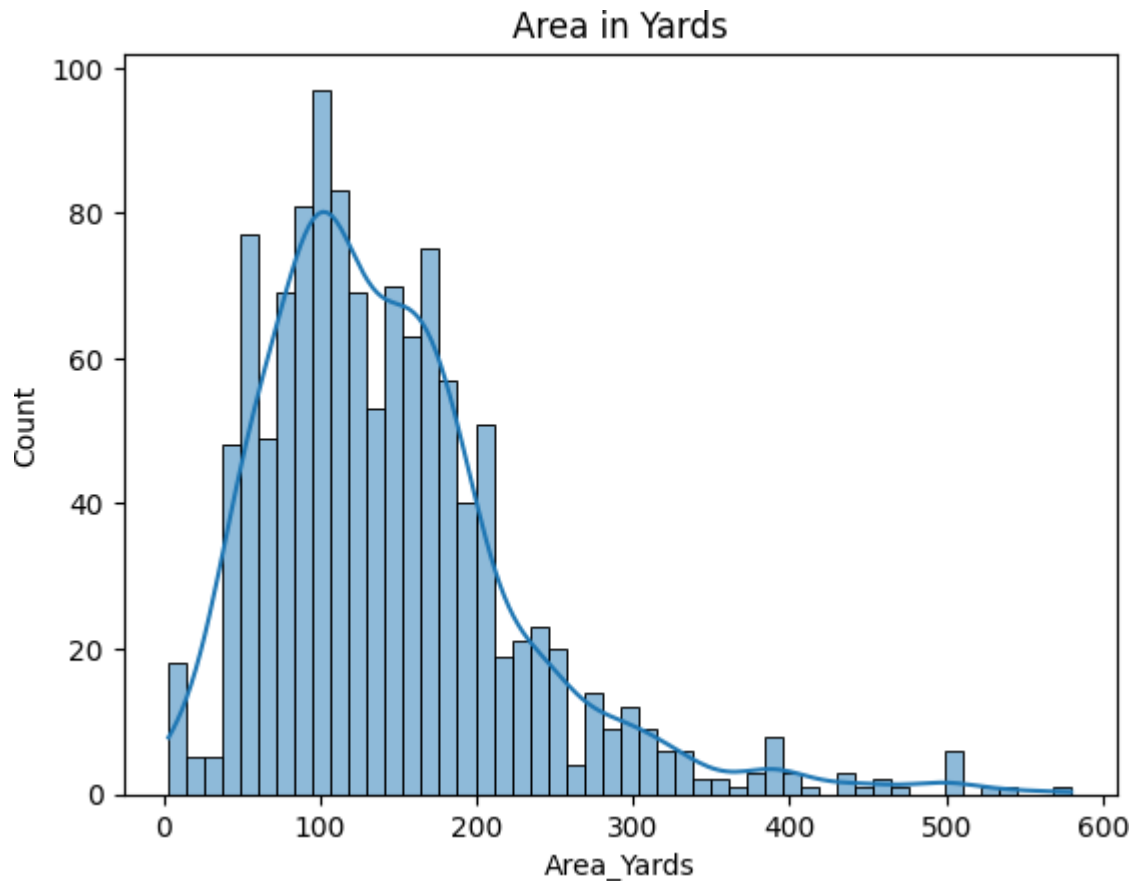
ExploratoryDataAnalysis

In the exploratory data analysis, I will be looking at the data and try to undersatnd the data. I will begin by looking at the distribution of data across the dataset, followed by visualizingthedataunderstandtherelationshipbetweenthefeaturesandthetarget variable.

AreaofHouses

```
In[ ]:sns.histplot(x=df['Area_Yards'],kde=True,bins=50).set_title('AreainYa
```

```
Out[ ]:Text(0.5,1.0, 'AreainYards')
```

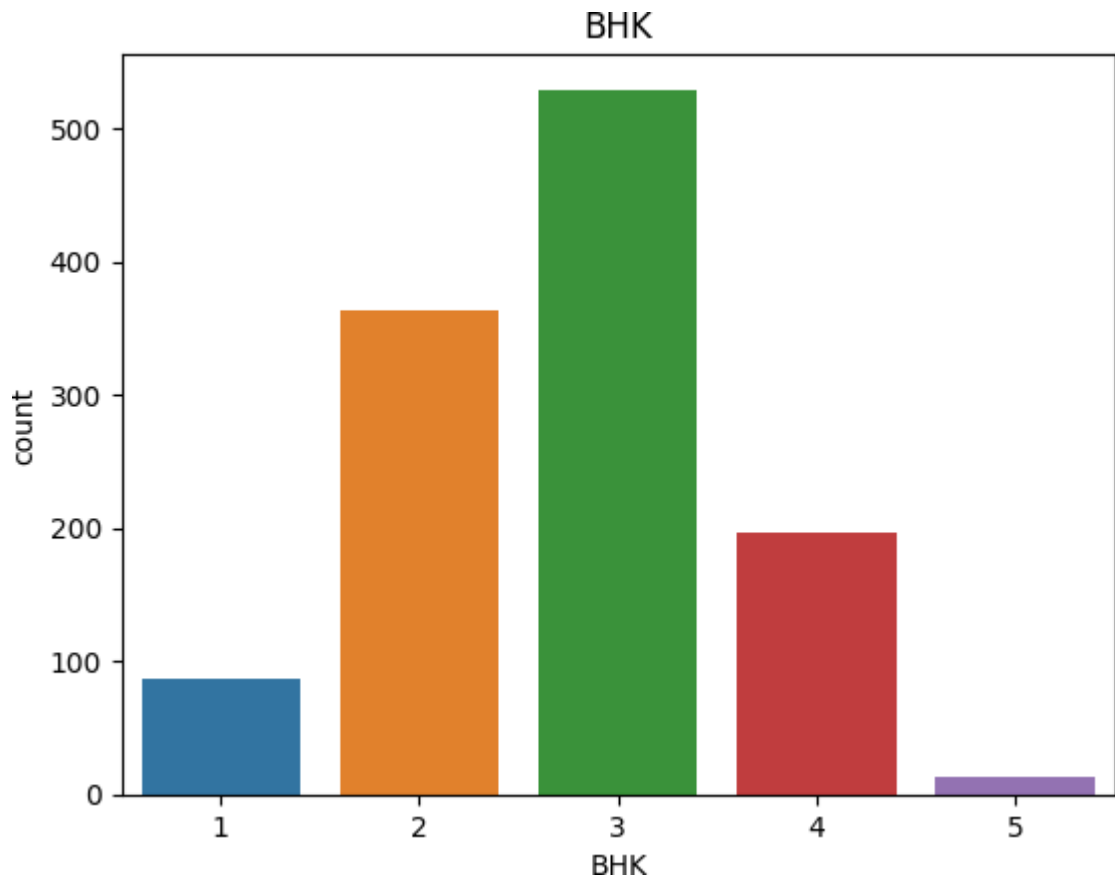



Looking at the distribution of Area if houses in delhi most of the houses have area between 80-200sq.yards. This means that most of the houses in delhi are small houses and there are few house having area near 300 sq yards. Whereas there are very few houses having area more than 400 sq yards. This representation helps us to know about availability of space in delhi.

BHK

```
In[]: sns.countplot(x='BHK',data=df).set_title('BHK')
```

```
Out[]:Text(0.5,1.0,'BHK')
```



BHK-BedroomHallKitchen.Fromthisgraphwecangetalittleideaaboutthedesignof most of houses in delhi. Most of houses are 3 BHK followed by 2, 4, 1 and 5 BHK houses.

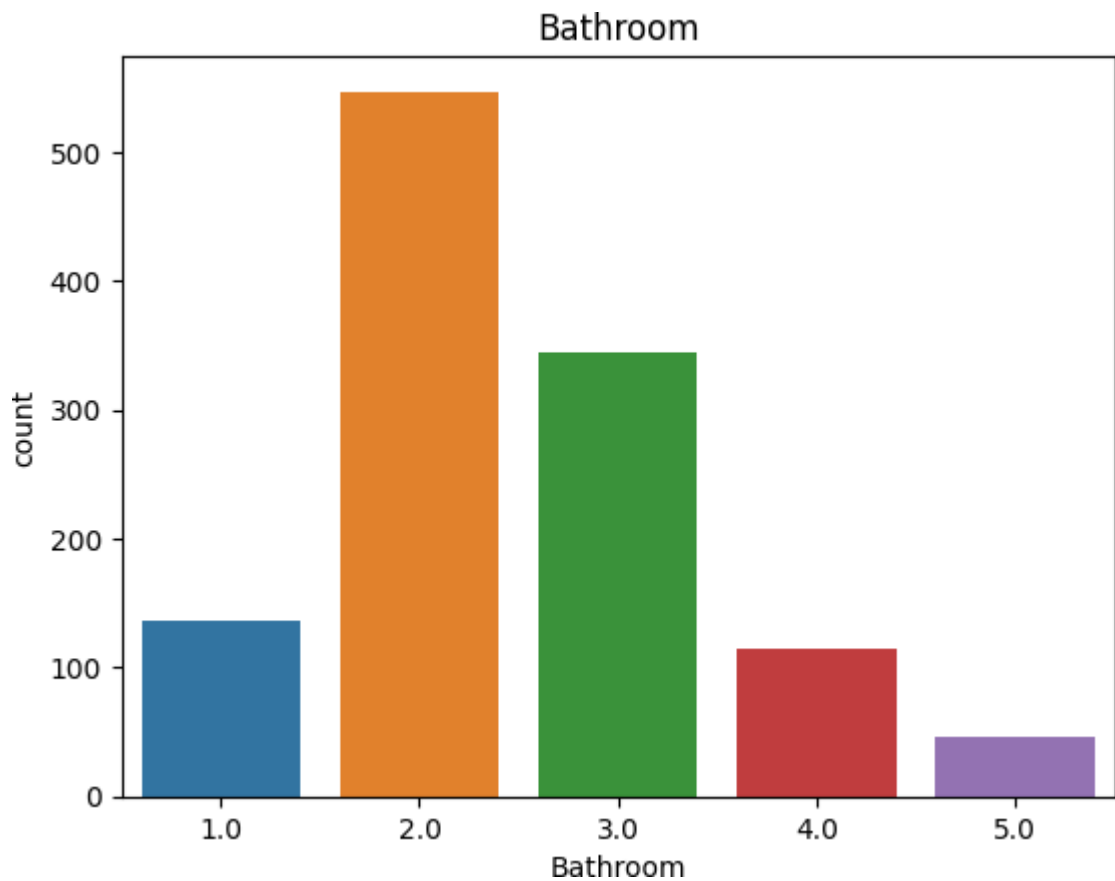
Majorityofthehouseshaveareabetween80-200sq.yards.Houseswithareanear200 yards can have maximum 3 bedrooms, for houses with area near 100 can have 2

bedroomsand1bedroomforhouseswitharealessthan80sqyards.Moreoverwhoses having area more than 300 are less in number so the count of 4BHK and 5BHK. Thus, thereisarelationbetweentheBHKandareaofhouse,withthisrelationwecangeta idea about the structure of houses.

BathroomCount

```
In[]: sns.countplot(x='Bathroom',data=df).set_title('Bathroom')
```

```
Out[]:Text(0.5,1.0, 'Bathroom')
```

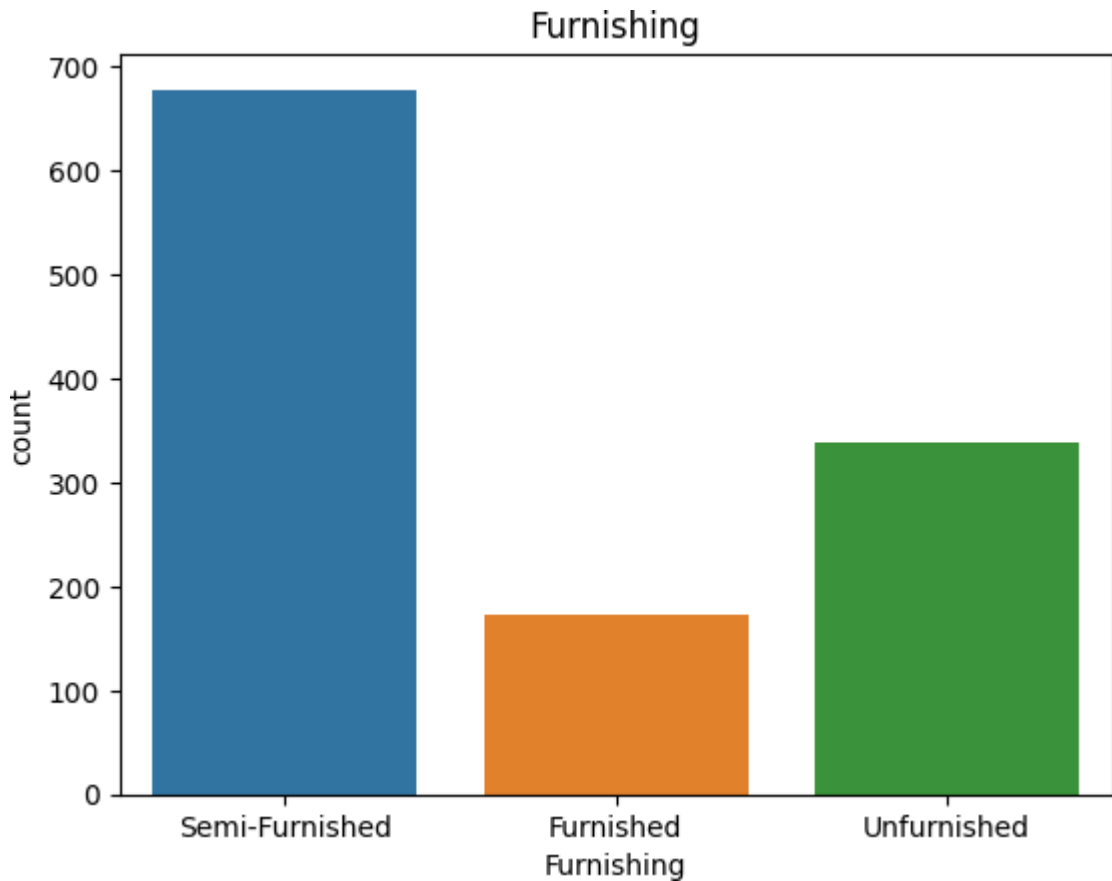


Majority of the houses have 2 bathrooms which is quite obvious. However, there are some houses having 3 or more than 3 bathrooms, which means these houses are quite big and spacious to have 3 or more bathrooms. Smaller houses usually have 1 bathroom.

Furnishing

```
In[]: sns.countplot(x='Furnishing', data=df).set_title('Furnishing')
```

```
Out[]: Text(0.5, 1.0, 'Furnishing')
```

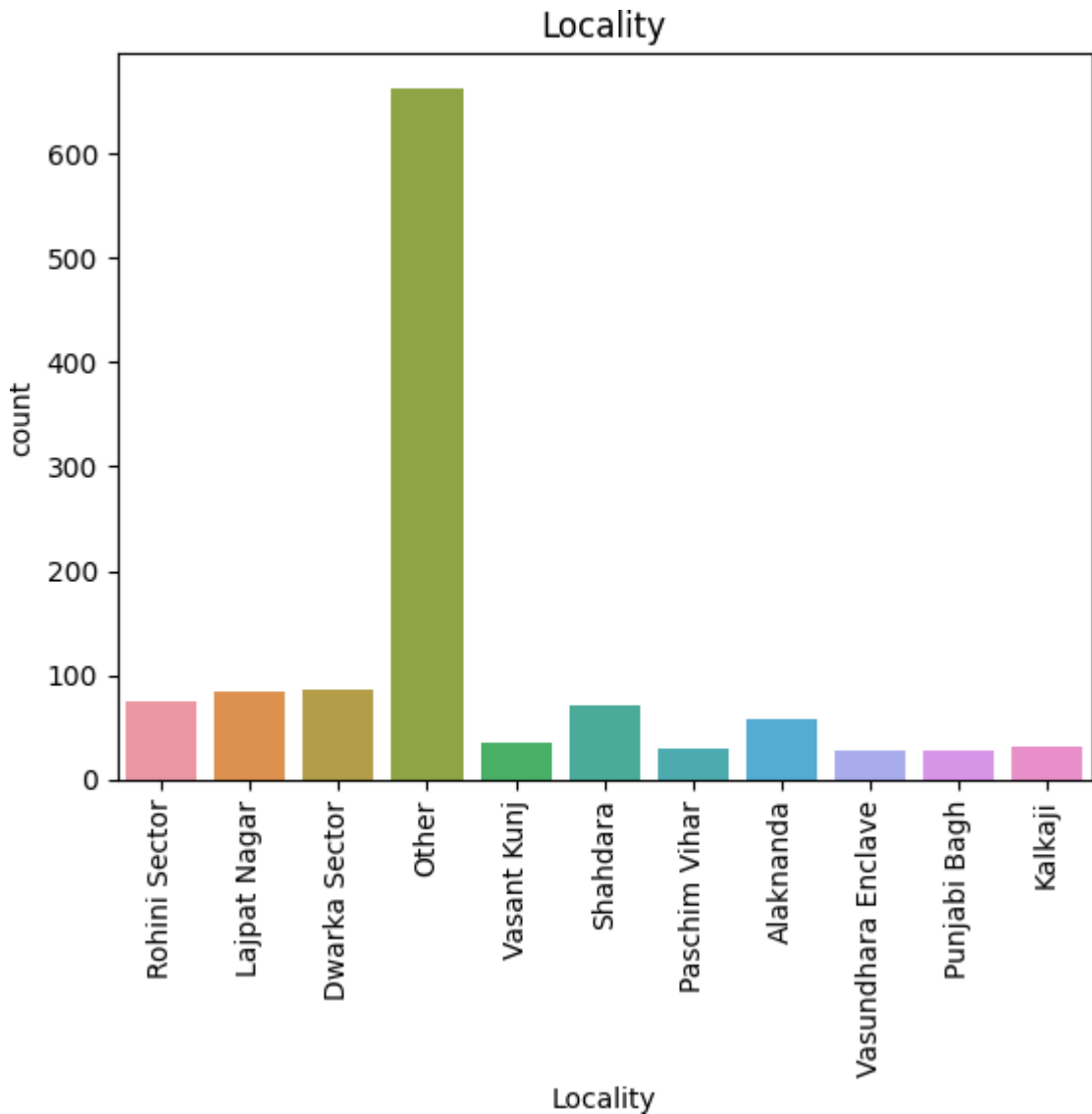


Delhi is very close capital of India, New Delhi and many people migrate from various regions of the country mostly for employment reasons to this region. As these people look for accommodations, the furnishing of the houses plays a major role. People who are migrating from huge distances cannot afford to move their furniture and other household items. Hence, they look for fully furnished houses. Similarly the people from neighbouring states might prefer semi-furnished houses, whereas Delhi locals who are moving to some other reason might prefer unfurnished houses. Hence, the furnishing of the house plays a major role in the price of the house. Hence, we have included this feature in our dataset

Locality

```
In[ ]: sns.countplot(x = 'Locality', data = df).set_title('Locality')
plt.xticks(rotation = 90)
```

```
Out[ ]: (array([0,1,2,3,4,5,6,7,8,9,10]),
 [Text(0, 0, 'Rohini Sector'),
  Text(1, 0, 'Lajpat Nagar'),
  Text(2,0,'DwarkaSector'),
  Text(3, 0, 'Other'),
  Text(4, 0, 'VasantKunj'),
  Text(5, 0, 'Shahdara'),
  Text(6, 0, 'PaschimVihar'),
  Text(7, 0, 'Alaknanda'),
  Text(8, 0, 'VasundharaEnclave'),
  Text(9, 0, 'PunjabiBagh'),
  Text(10,0,'Kalkaji')])
```

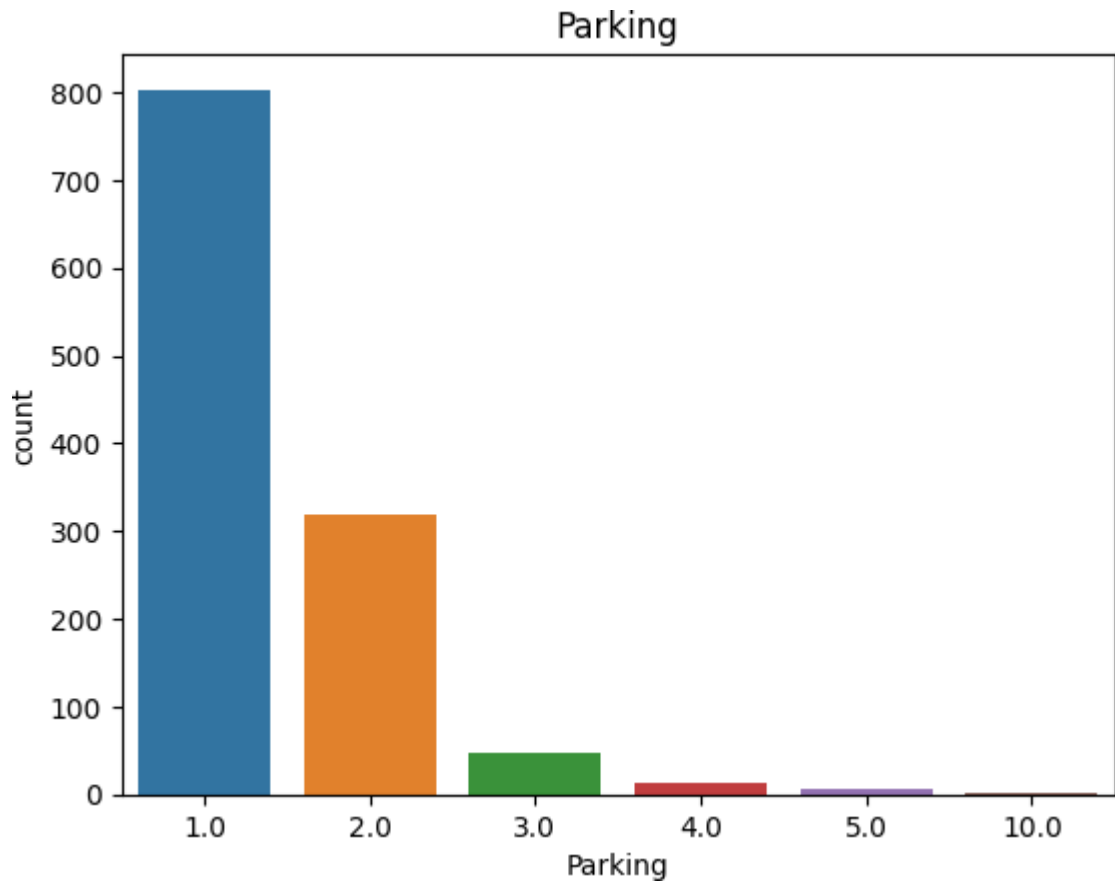


Since there are so many localities and addresses in the dataset, I have grouped nearly half of them in the top ten localities (count wise), and the remaining localities are grouped as 'Others'. Upon visualizing the locality on the graph, we can see that after the 'Other' category, the Dwarka Sector has the highest number of houses followed by Lajpat Nagar and Rohini Sector. From this info, I assume that these localities are good to settle in Delhi. In addition to that localities such as Shahdara and Alaknanda have a significant number of houses as well. So, these localities are also good to settle in Delhi.

Parking

```
In[ ]: sns.countplot(x='Parking', data=df).set_title('Parking')
```

```
Out[ ]: Text(0.5, 1.0, 'Parking')
```

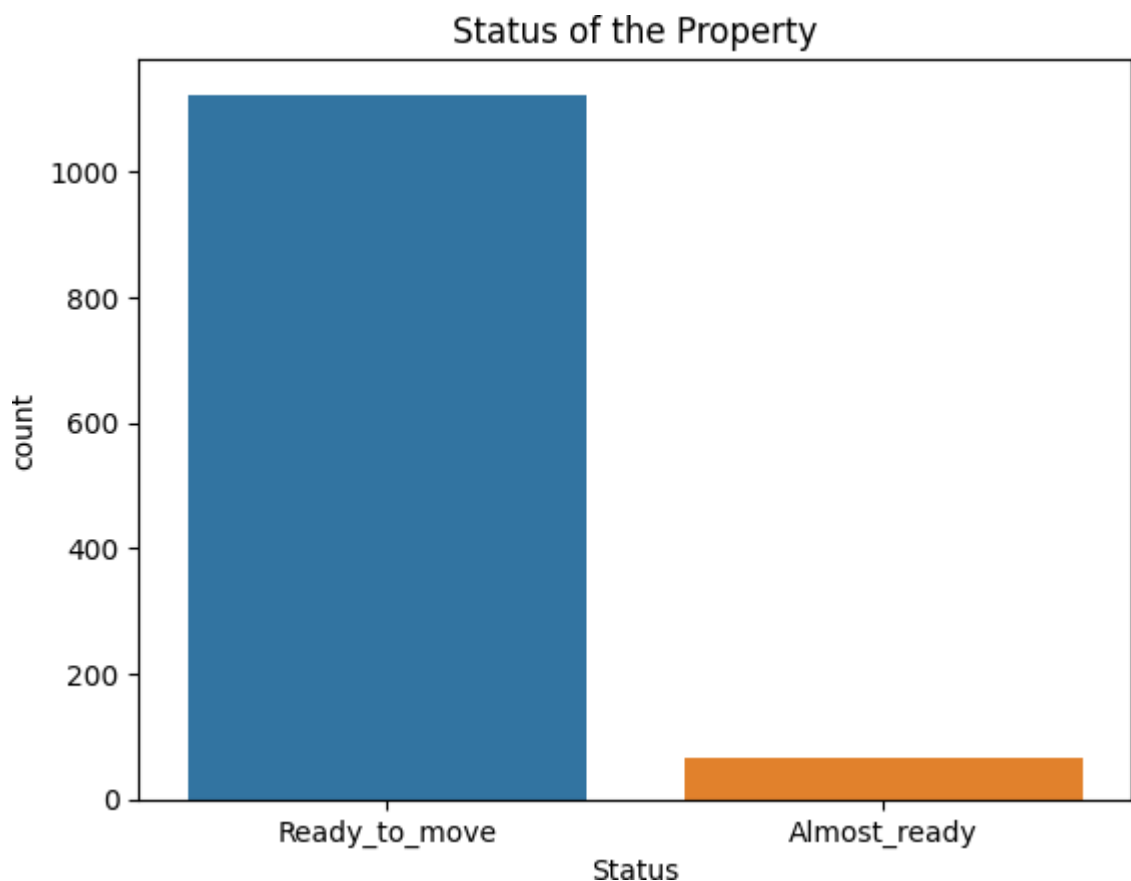


Majority of the houses in Delhi has one car parking which is quite common. Few of the houses have enough space for two car parking and very few houses have more than two car parking space. We can relate this graph to the graph with house area, where majority of the houses have area between 100 -200 sq. yards. So, it is quite obvious that these houses will have one car parking space. The houses with area more than 200 sq. yards will have more than one car parking space.

Status

```
In[]: sns.countplot(x='Status',data=df).set_title('Status of the Property')
```

```
Out[]: Text(0.5,1.0, 'Status of the Property')
```

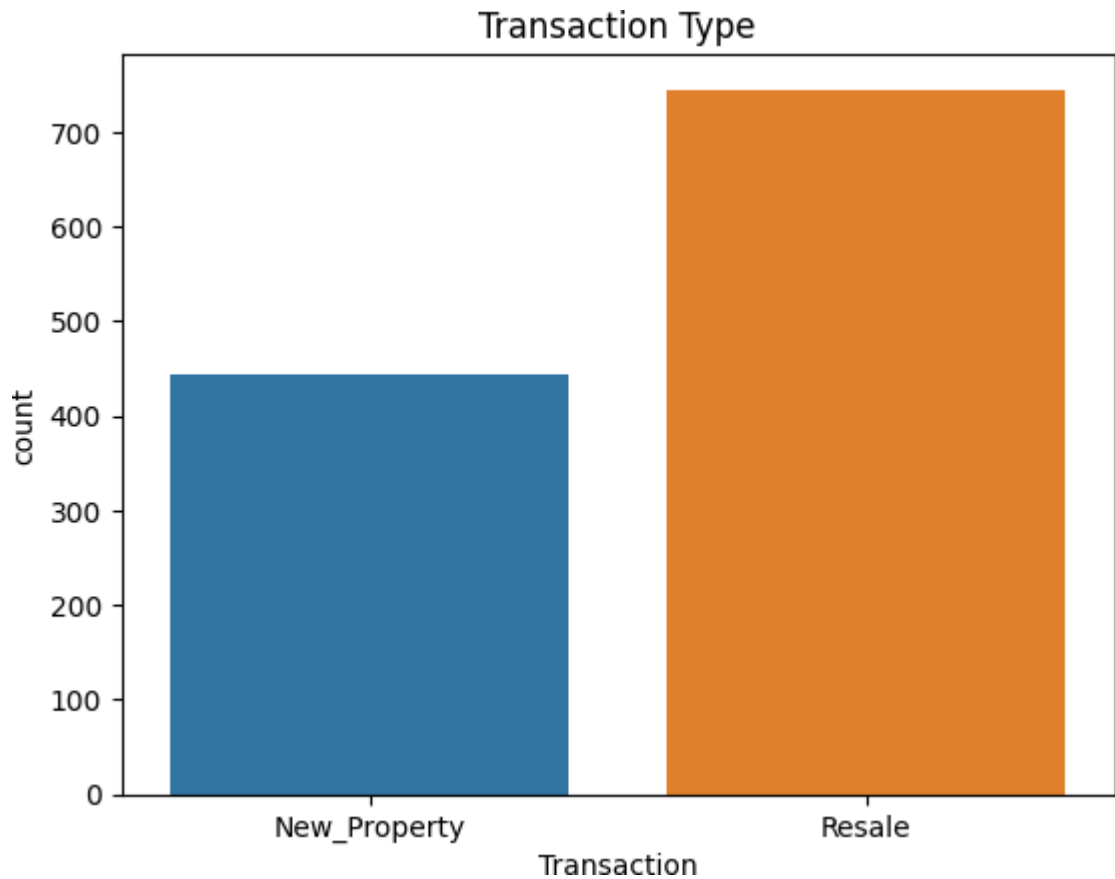


Most of the houses are ready to move and actively looking for buyers. Very few houses are still under construction and would be ready to move soon.

TransactionType

```
In[]: sns.countplot(x='Transaction', data=df).set_title('TransactionType')
```

```
Out[]: Text(0.5, 1.0, 'TransactionType')
```

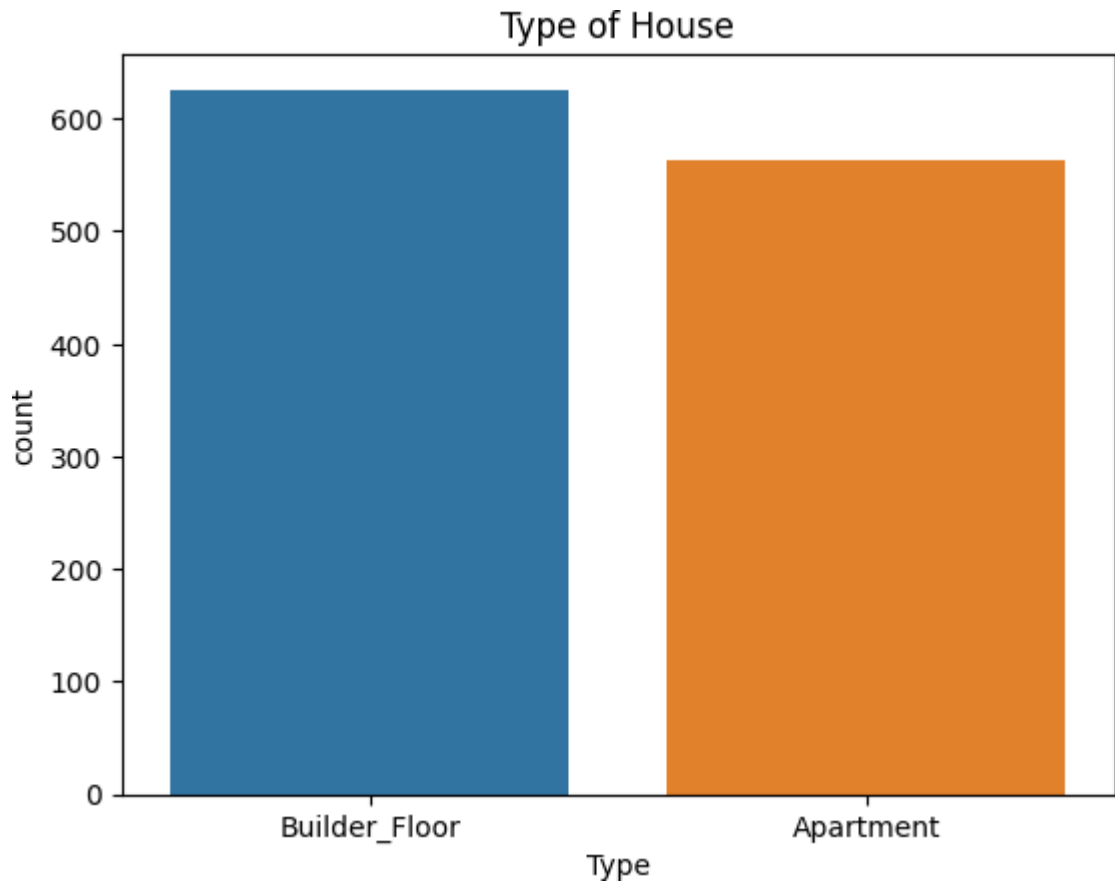


A huge number of houses are resale type, which means a lot of people are moving out of Delhi. This could be due to the high pollution levels or some other reason. This could be a good opportunity for people who are looking to buy a house in Delhi. Nearly 430 houses are new property houses which are built with only the purpose to be commercially sold.

House Type

```
In[ ]: sns.countplot(x='Type', data=df).set_title('Type of House')
```

```
Out[ ]: Text(0.5, 1.0, 'Type of House')
```

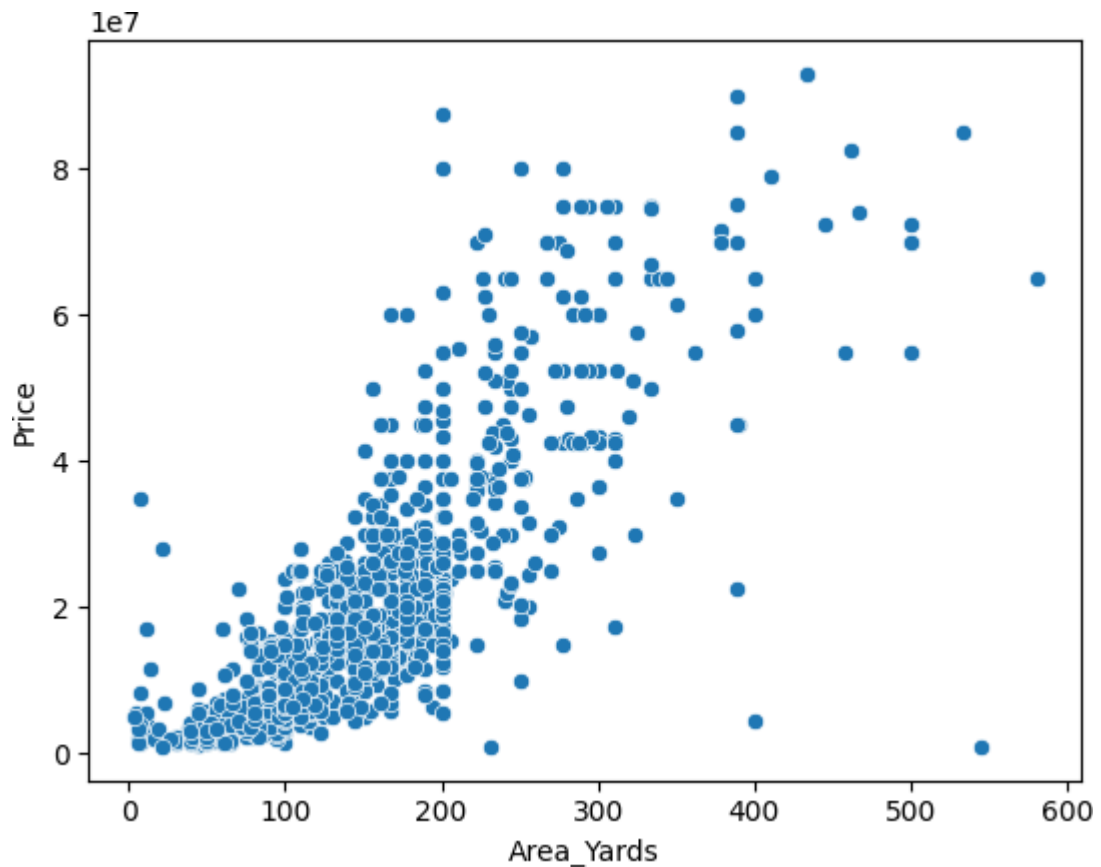
Most of the houses are builder floor which means people like to live in independent houses rather than apartments, due to the privacy and space they get in independent houses.

Till now, I have visualized the distribution of data across variables in the dataset. Now, I will be looking at the relation between the target variable i.e Price and independent variables.

Area and Price

```
In[]: sns.scatterplot(x='Area_Yards',y='Price',data=df)
```

```
Out[]: <Axes: xlabel='Area_Yards', ylabel='Price'>
```

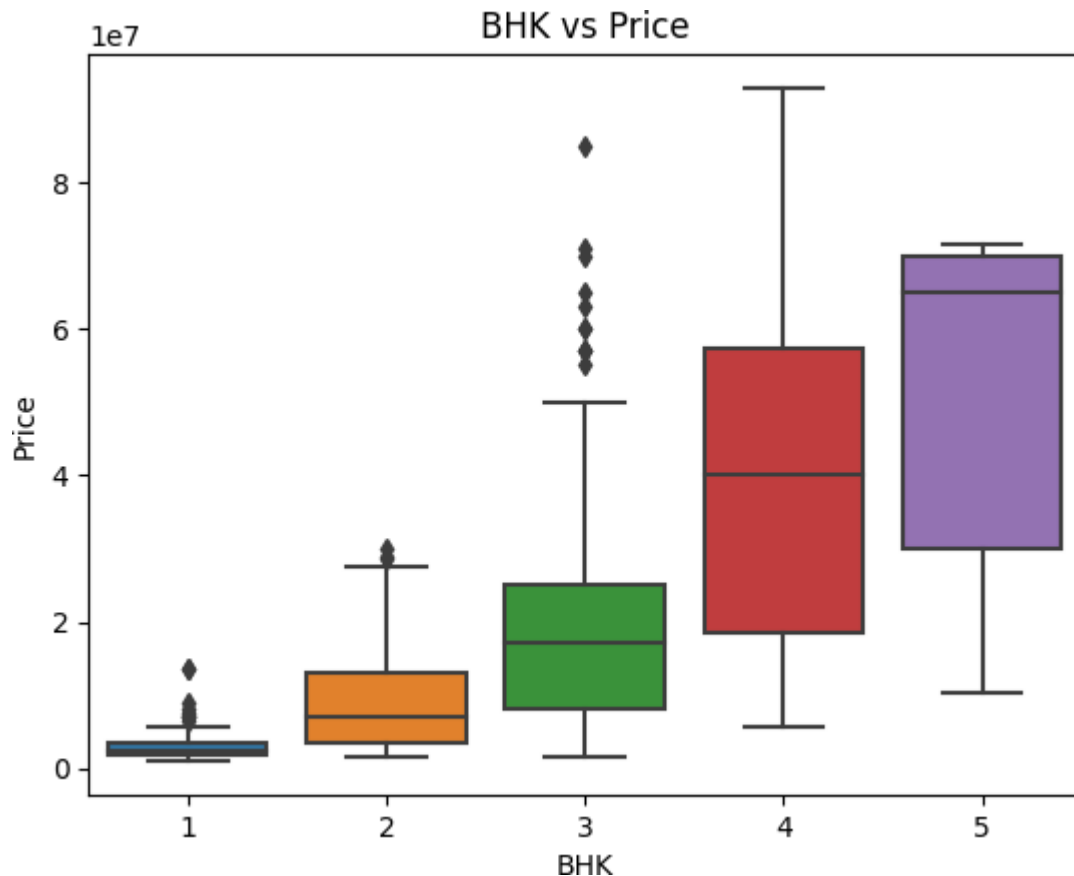


The scatterplot graph has trend of increase in price with increase in area, which is obvious. However, there are some houses whose price is lower as compared to other with similar area, which means there are several other factors which affect price of the house.

BHK and Price

```
In[ ]: sns.boxplot(x='BHK',y='Price',data=df).set_title('BHKvsPrice')
```

```
Out[ ]: Text(0.5,1.0, 'BHKvsPrice')
```

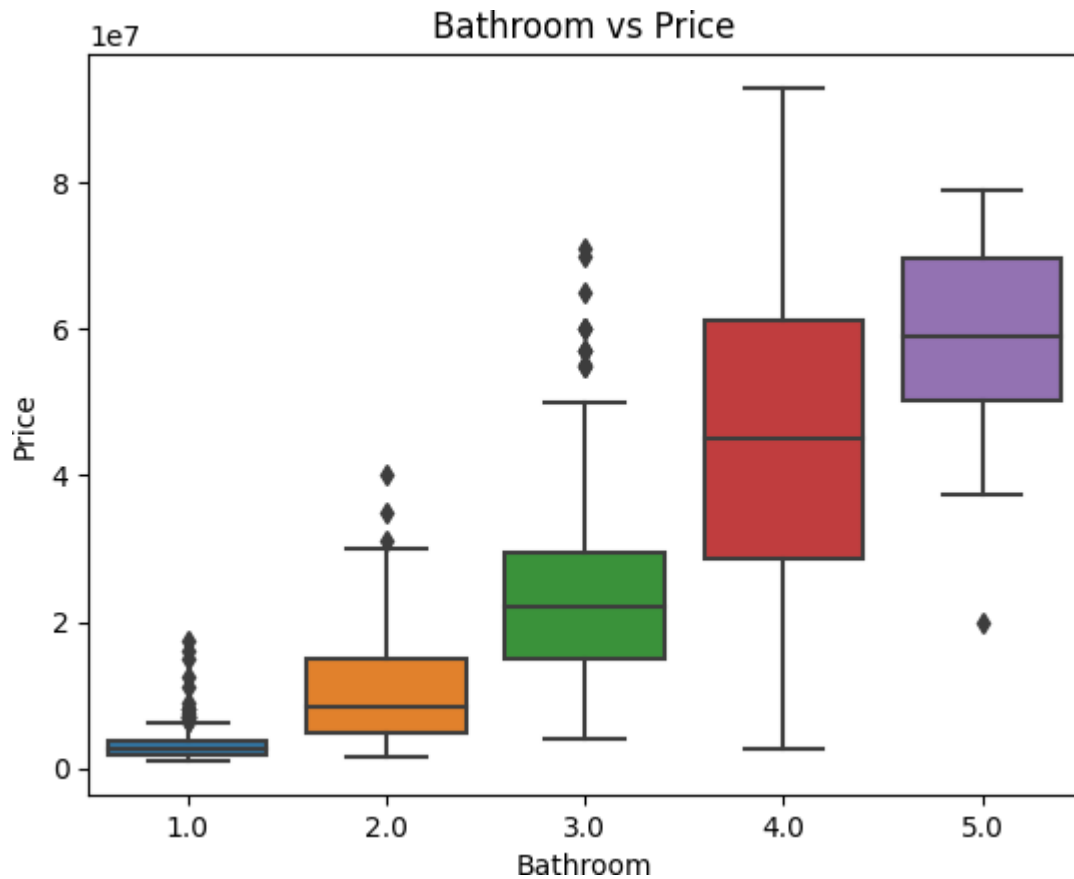


From this boxplot, we get to know about the relation between the price of the house and the BHK count. We can see that the price of the house increases with the increase in the BHK count, which is evident from the fact that 5 BHK houses have the highest median price i.e. nearly 7,00,00,000 INR followed by 4 BHK houses with a median price of nearly 4,00,00,000 INR. In comparison to that, 3 BHK houses have a median price near about 1 crore INR and 2 BHK houses have a median price of nearly 50,00,000 INR. The 1 BHK houses have the lowest median price of nearly 30,00,000 INR.

Bathroom count and Price

```
In[]: sns.boxplot(x='Bathroom', y='Price', data=df).set_title('Bathroom vs Price')
```

```
Out[]: Text(0.5, 1.0, 'Bathroom vs Price')
```

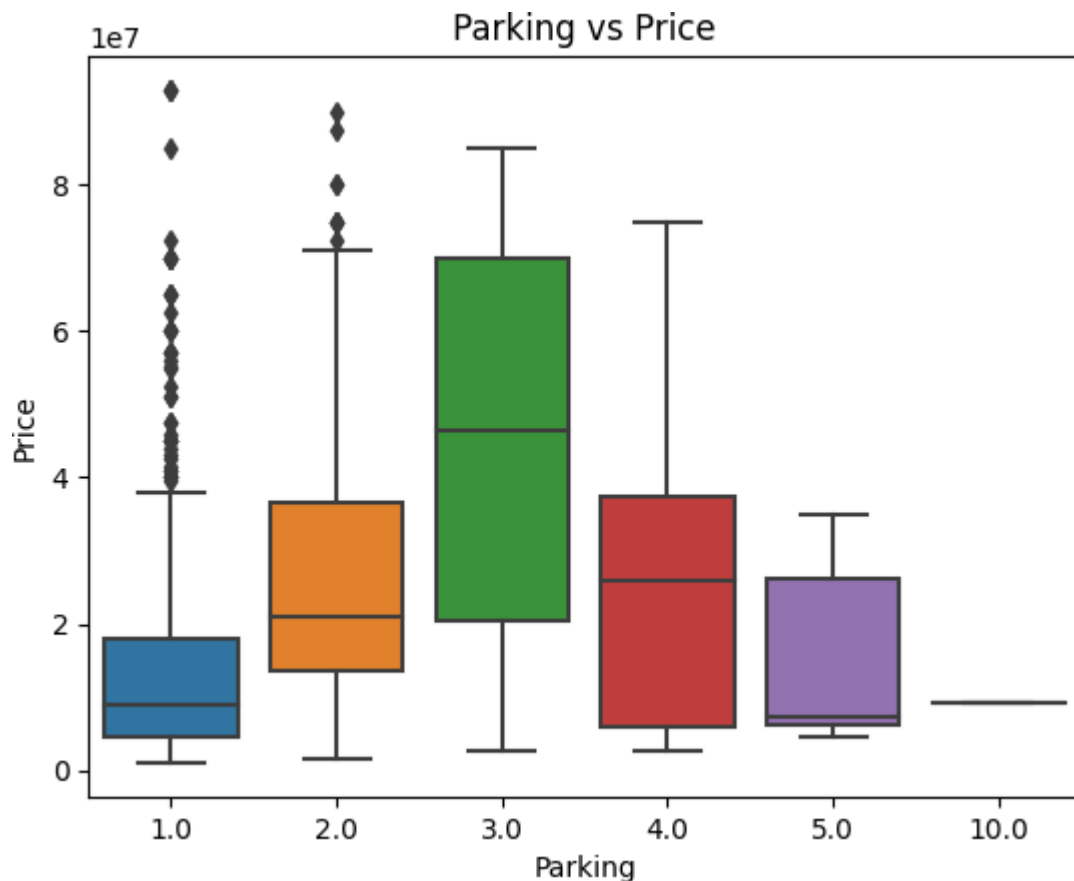


The graph is similar to the previous graph for BHK and Price. Here the price of the house increases with an increase in the number of bathrooms. Moreover, each bathroom count has a similar house price as the previous graph for BHK and Price. Therefore, we can say that the number of bathrooms and the BHK of the house are highly correlated.

Parking and Price

```
In[]: sns.boxplot(x='Parking', y='Price', data=df).set_title('Parking vs Price')
```

```
Out[]: Text(0.5, 1.0, 'Parking vs Price')
```

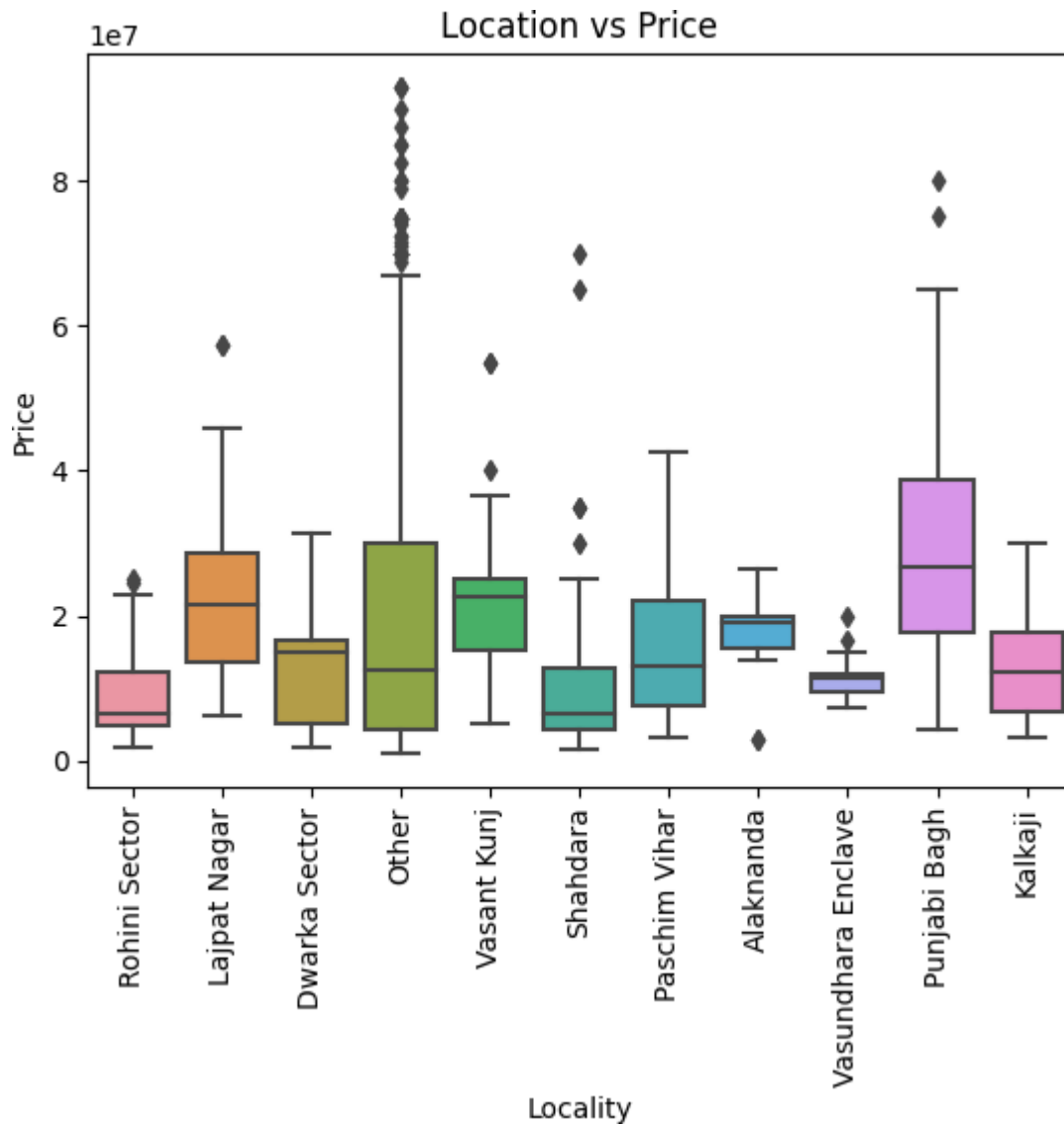


This box plot graph shows the relationship between the parking space and the price of the house. Houses with 3 parking spaces have the highest median price, which is 4,50,00,000, followed by 2 parking spaces, 1 parking space, 4 parking spaces, and 5 parking spaces. From this graph, we can assume that people usually look for houses to have sufficient parking space, which means they don't want more than enough space for parking, which is evident through the lower median price of houses with 4 parking spaces or more.

Locality and Price

```
In[]: sns.boxplot(x='Locality', y='Price', data=df).set_title('Location vs Price')
plt.xticks(rotation=90)
```

```
Out[]: (array([0,1,2,3,4,5,6,7,8,9,10]),
 [Text(0, 0, 'Rohini Sector'),
  Text(1, 0, 'Lajpat Nagar'),
  Text(2,0, 'DwarkaSector'),
  Text(3, 0, 'Other'),
  Text(4, 0, 'VasantKunj'),
  Text(5, 0, 'Shahdara'),
  Text(6, 0, 'PaschimVihar'),
  Text(7, 0, 'Alaknanda'),
  Text(8, 0, 'VasundharaEnclave'),
  Text(9, 0, 'PunjabiBagh'),
  Text(10,0,'Kalkaji')])
```

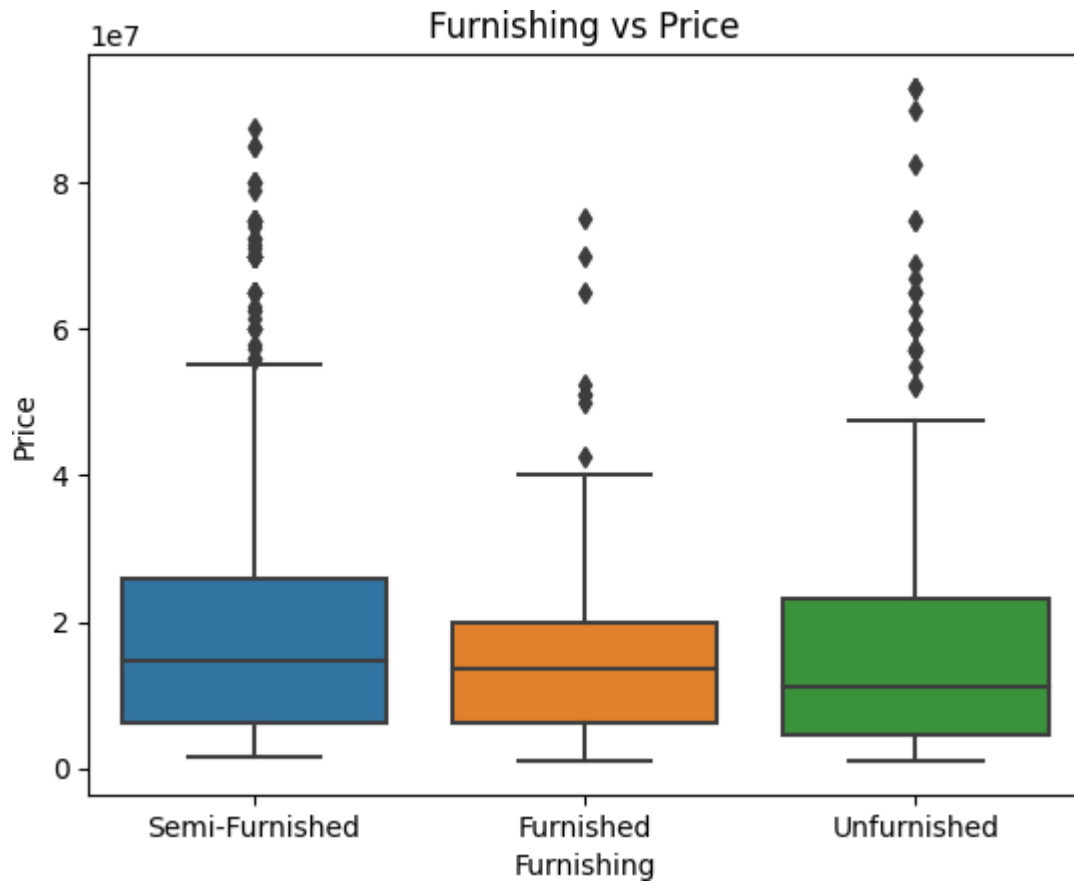


From this graph, we can get an idea about the localities along with the house price. Punjabi Bagh locality has the highest median price of nearly 2,50,00,000 INR, which means this is a posh locality. Punjabi Bagh is followed by Lajpat Nagar and Vasant Kunj. These two localities can be included in the posh locality. The localities with lowest median prices include - Rohini Sector, Vasundhara Enclave and Shahdara. Some of the mediocre localities are - Dwarka Sector, Paschim Vihar, Kalkaji, and the rest of the localities are average.

Furnishing and Price

```
In[]: sns.boxplot(x='Furnishing', y='Price', data=df).set_title('Furnishing vs P
```

```
Out[]: Text(0.5, 1.0, 'Furnishing vs Price')
```

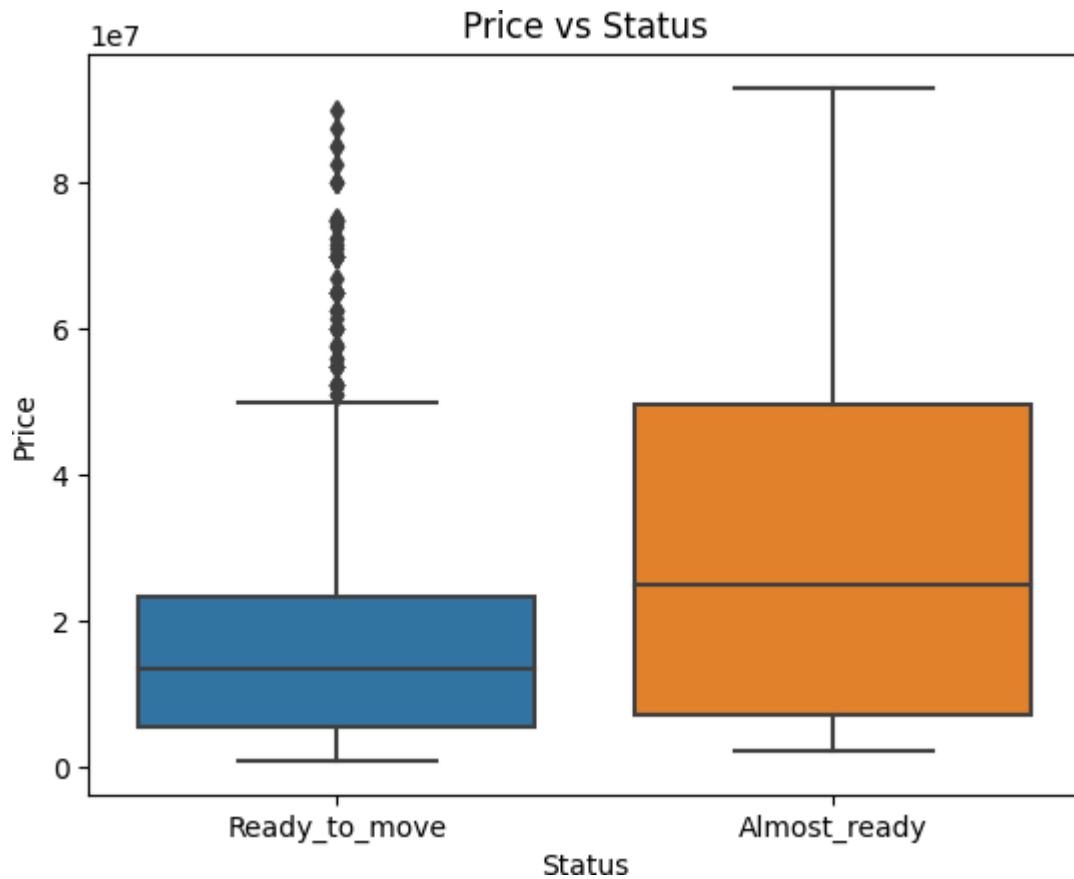


There is very little difference in the median house price based on the furnishing status. Interestingly, the furnished houses have a lower median price than the semi-furnished houses. The unfurnished houses have the lowest median price.

Status and Price

```
In[]: sns.boxplot(x='Status',y='Price',data=df).set_title('Price vs Status')
```

```
Out[]: Text(0.5,1.0, 'Price vs Status')
```

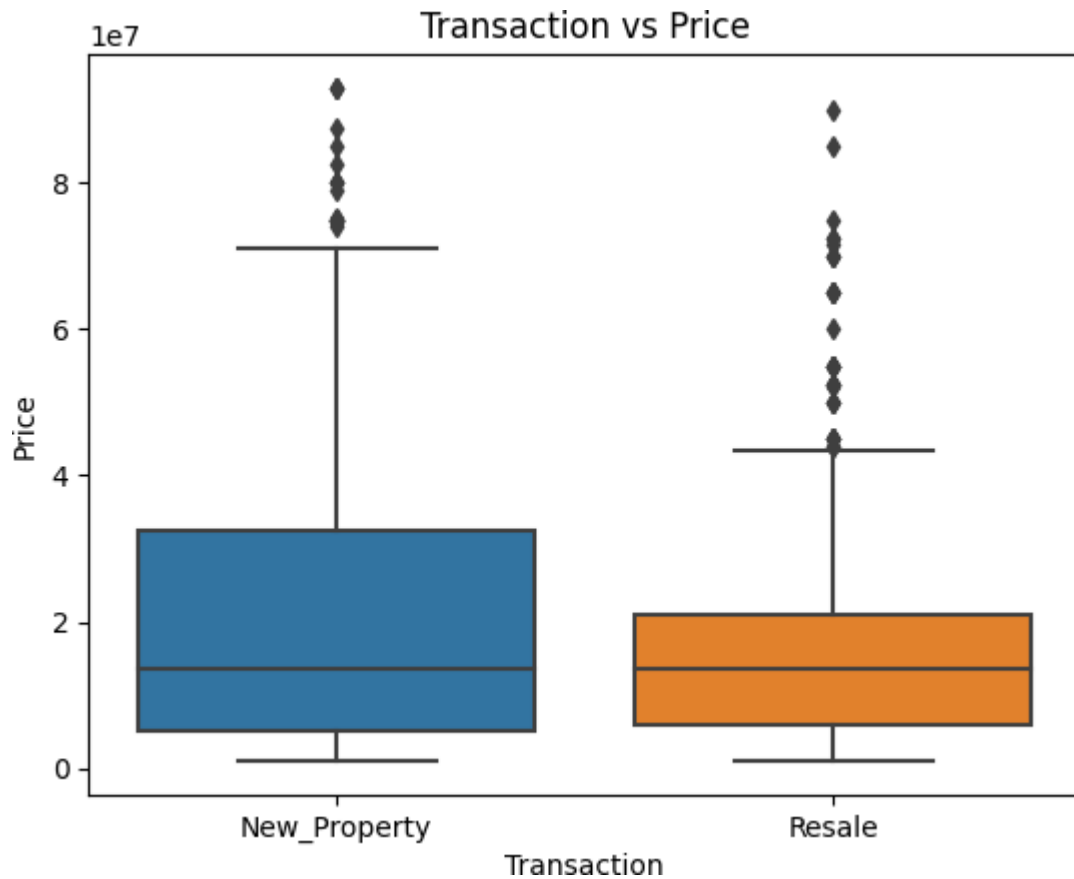


Surprisingly the houses that are still under construction have higher median price than those which are ready to move in. This might be because the houses that are still under construction allow the buyers to make changes to interior/exterior.

Transaction Type and Price

```
In[]: sns.boxplot(x='Transaction', y='Price', data=df).set_title('Transaction vs
```

```
Out[]: Text(0.5, 1.0, 'Transaction vs Price')
```

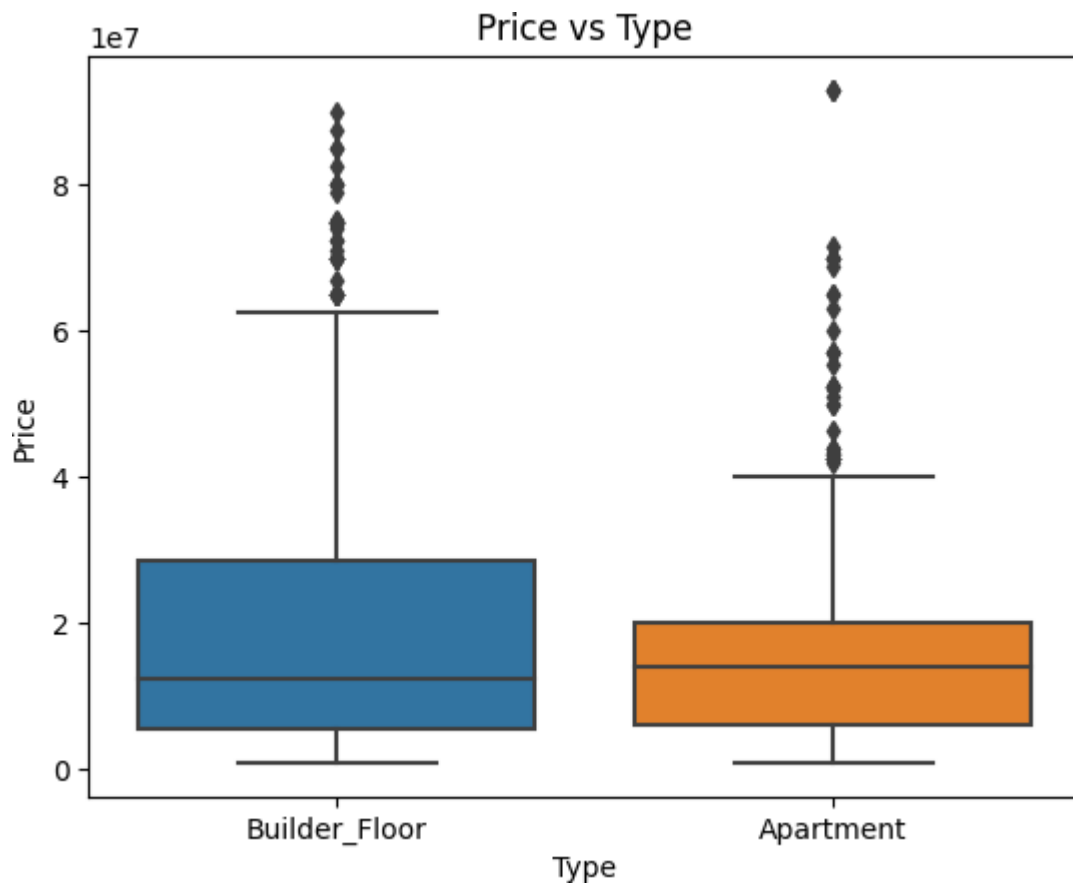



As expected the new properties have higher price than old ones/ resale ones. The new properties attracts more buyers to make features including reliability, designs. Whereas the resale one sometimes poses doubts for the buyers.

Property Type and Price

```
In[ ]: sns.boxplot(x='Type', y='Price', data=df).set_title('Price vs Type')
```

```
Out[ ]: Text(0.5, 1.0, 'Price vs Type')
```



Both the Builder Floor and Apartment type houses have nearly the same median price with Apartment type houses having slightly higher median price. However, the builder floor type houses are more in number which means people are more interested in buying builder floor type houses.

Data Preprocessing 2

Label encoding the categorical variables

```
In[ ]: from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

#Columns for label encoding
cols=['Furnishing','Locality','Status','Transaction','Type']

for i in cols:
    le.fit(df[i])
    df[i] = le.transform(df[i])
    print(i, df[i].unique())
```

```
Furnishing[102]
Locality[731498501062]
Status[10]
Transaction[01]
Type[10]
```

Normalizing the continuous features

```
In[]: from sklearn.preprocessing import MinMaxScaler
min_max = MinMaxScaler()
df[['Area', 'Price', 'Per_Sqft', 'Area_Yards']] = MinMaxScaler().fit_transform(d
```

```
In[]: df.head()
```

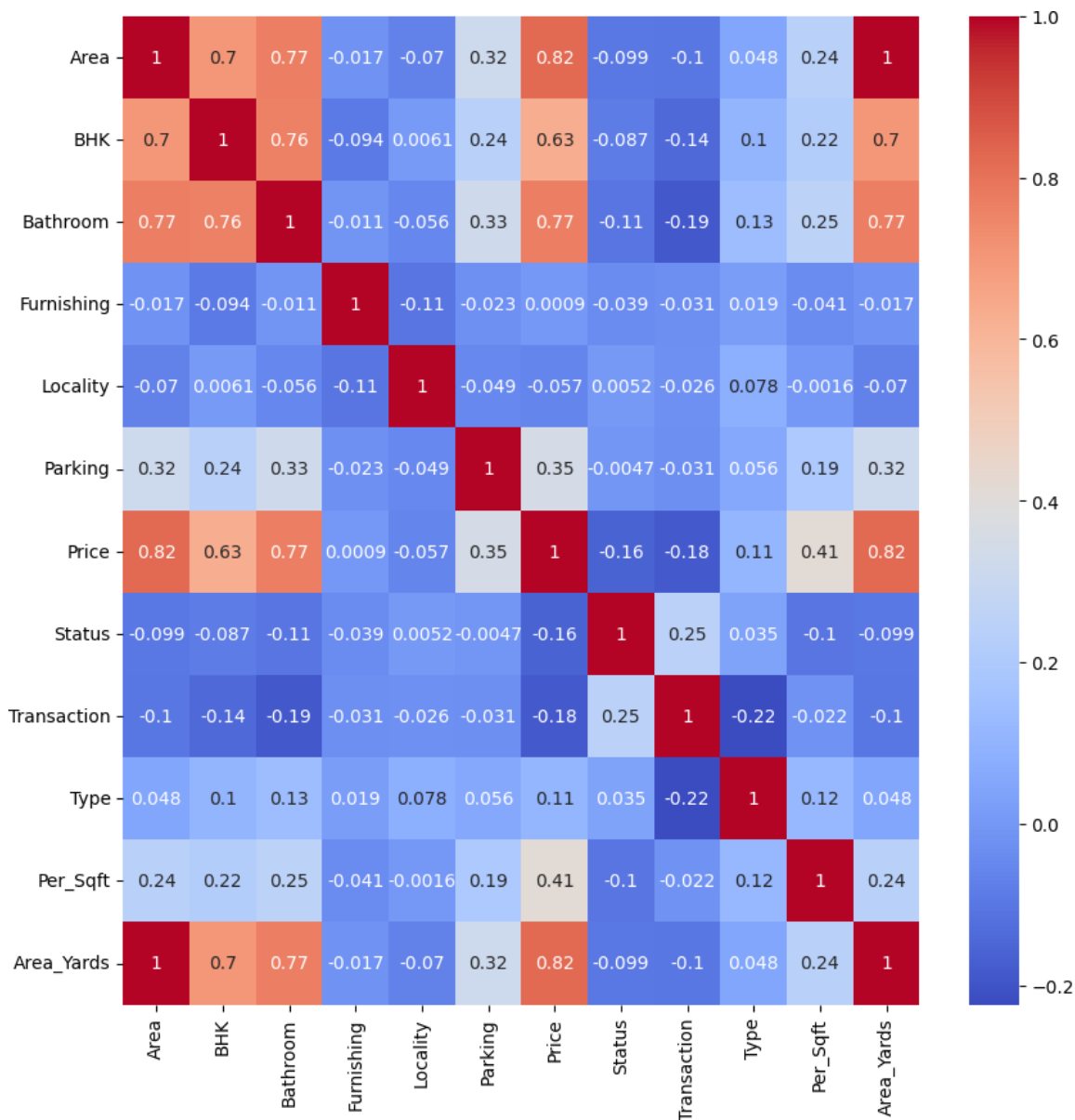
Out[]:

	Area	BHK	Bathroom	Furnishing	Locality	Parking	Price	StatusTransacti
	00.148690	3	2.0	1	7	1.0	0.059783	1
	10.139060	2	2.0	1	7	1.0	0.043478	1
	20.177581	2	2.0	0	7	1.0	0.157609	1
	30.110169	2	2.0	1	7	1.0	0.034783	1
	40.119800	2	2.0	1	7	1.0	0.056522	1

CoorelationMatrix Heatmap

```
In[ ]: plt.figure(figsize=(10,10))
sns.heatmap(df.corr(),annot=True,cmap='coolwarm')
```

```
Out[ ]: <Axes:>
```



In this coorelation matrix heatmap, we can see that the price of the house has high positivecoorelationwithlandarea,BHK,bathroomcount,whichprovesareprevious obersevation about their relation.

TrainTestSplit

```
In[]: #DroppingPer_Sqftcolumn
df.drop(['Per_Sqft'],axis=1,inplace=True)
```

```
In[]: fromsklearn.model_selectionimporttrain_test_split
X_train,X_test,y_train,y_test=train_test_split(df.drop('Price',axis=1),df['
```

Price Prediction

I will be using the following models: •

- Decision Tree Regressor
- Random Forest Regressor

DecisionTreeRegressor

```
In [ ]: from sklearn.tree import DecisionTreeRegressor

#creating DecisionTreeRegressor object
dtr = DecisionTreeRegressor()
```

Hypertuning the model with GridSearchCV

```
In [ ]: from sklearn.model_selection import GridSearchCV

#Defining parameters
parameters = {'max_depth': [2, 4, 6, 8],
              'min_samples_split': [2, 4, 6, 8],
              'min_samples_leaf': [1, 2, 3, 4],
              'max_features': ['auto', 'sqrt', 'log2'],
              'random_state': [0, 42]}

#Creating GridSearchCV object
grid_search = GridSearchCV(dtr, parameters, cv=5, scoring='neg_mean_squared_error')

#Fitting data to grid search object
grid_search.fit(X_train, y_train)

#Best parameters
print("Best parameters:", grid_search.best_params_)
```

Best parameters: {'max_depth': 6, 'max_features': 'auto', 'min_samples_leaf': 1, 'min_samples_split': 8, 'random_state': 42}

```
In [ ]: dtr = DecisionTreeRegressor( max_depth=6, max_features='auto', min_samples_leaf
dtr
```

```
Out [ ]: ▼ DecisionTreeRegressor
DecisionTreeRegressor(max_depth=6, max_features='auto', min_samples_split=8,
                      random_state=42)
```

```
In [ ]: #Training the model
dtr.fit(X_train, y_train)
```

Out[]:

```
DecisionTreeRegressor
DecisionTreeRegressor(max_depth=6,max_features='auto',min_samples_split=8,
                      random_state=42)
```

In[]:

```
#TrainingAccuracy
dtr.score(X_train,y_train)
```

Out[]: 0.8545210312800097

In[]:

```
#Predictingthehouseprice
d_pred=dtr.predict(X_test)
```

Evaluating DecisionTreeRegressor Model

In[]:

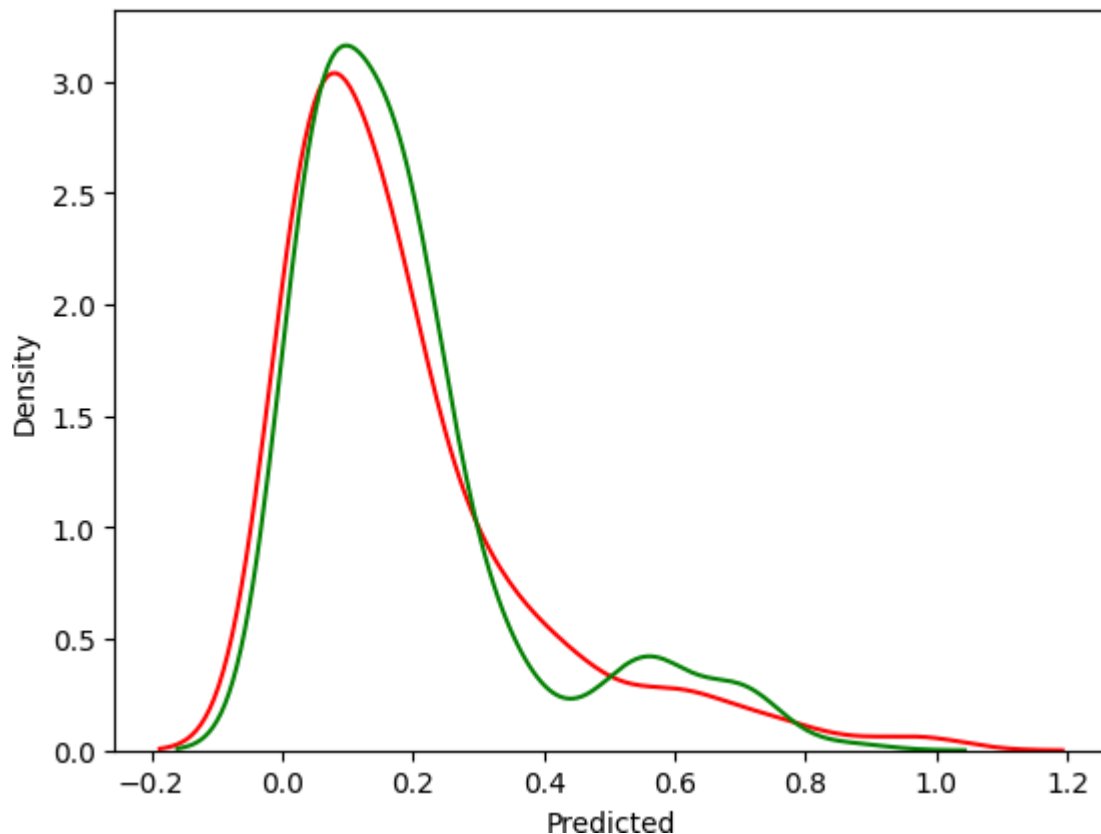
```
dft = pd.DataFrame({'Actual': y_test, 'Predicted': d_pred})
dft.reset_index(drop=True, inplace=True)
dft.head(10)
```

Out[]:

	Actual	Predicted
00	190217	0.033967
10	597826	0.561394
20	076087	0.102298
30	009239	0.018041
40	028261	0.018041
50	641304	0.584239
60	217391	0.214022
70	157609	0.171645
80	695652	0.472826
90	039348	0.067916

```
In[ ]: ax=sns.distplot(dft['Actual'],color='r',label='ActualPrice',hist=False)
sns.distplot(dft['Predicted'],color='g',label='PredictedPrice',ax=ax,hi
```

Out[]: <Axes: xlabel='Predicted', ylabel='Density'>



The red shows the distribution count for actual values and the green shows the distribution count for predicted values. The predicted value line tries to follow the actual value line as closely as possible. The closer the two lines are, the better the model is at predicting the house prices.

```
In[ ]: from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
print("R2 Score: ", r2_score(y_test, d_pred))
print("MeanSquaredError:", mean_squared_error(y_test, d_pred))
print("MeanAbsoluteError:", mean_absolute_error(y_test, d_pred))
print("RootMeanSquaredError:", np.sqrt(mean_squared_error(y_test, d_pred)))
```

```
R2Score:0.829160558769424
Mean Squared Error:0.006271711839596123
MeanAbsoluteError:0.05365863521857312
RootMeanSquaredError:0.07919414018471393
```

RandomForestRegressor

```
In[ ]: from sklearn.ensemble import RandomForestRegressor

# RandomForestRegressor Object
rfr = RandomForestRegressor()
rfr
```

```
Out[ ]: ▼ RandomForestRegressor
RandomForestRegressor()
```

```
In[ ]: # Training the model
rfr.fit(X_train, y_train)
```

```
Out[: ▼RandomForestRegressor
RandomForestRegressor()
```

```
In[: #TrainingAccuracy
rfr.score(X_train,y_train)
```

```
Out[: 0.962961816363294 In
```

```
[ ]: #Predictingthehouseprice
r_pred=rfr.predict(X_test)
```

EvaluatingtheRandomForestRegressorModel

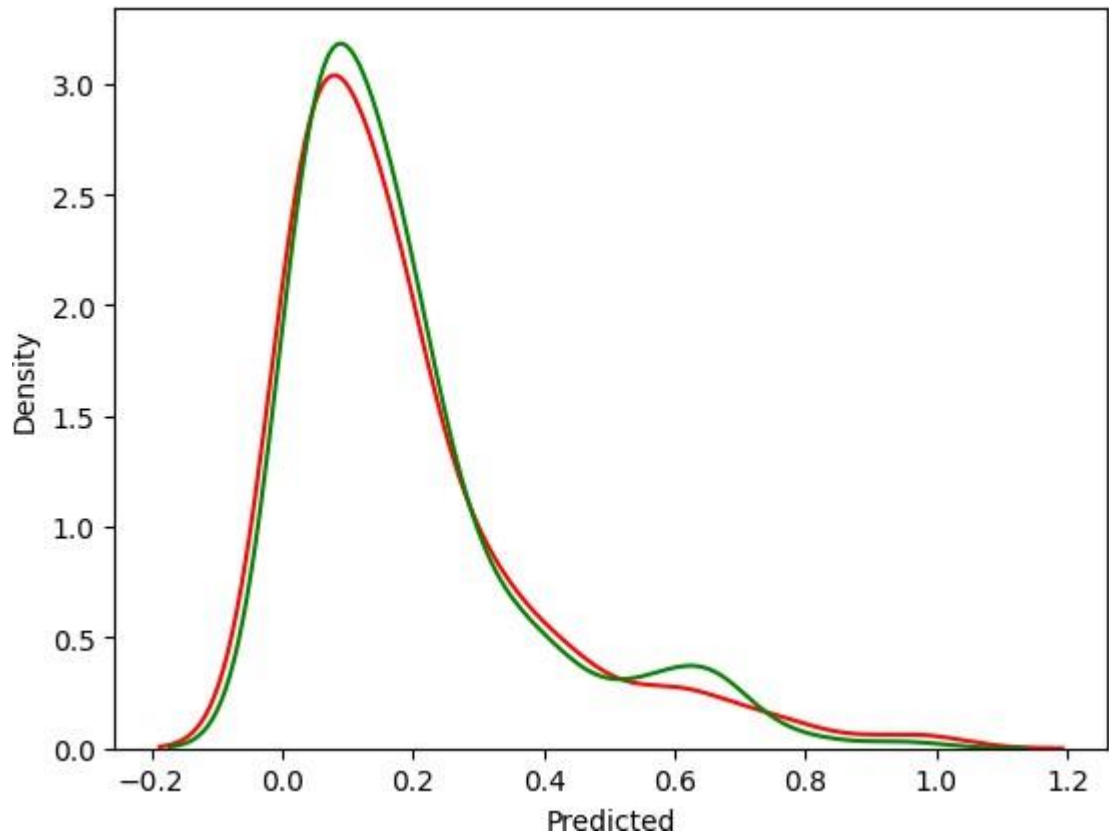
```
In[: dfr = pd.DataFrame({'Actual': y_test, 'Predicted': r_pred})
dfr.reset_index(drop=True, inplace=True)
dfr.head(10)
```

```
Out[:
```

	Actual	Predicted
00	190217	0.145777
10	597826	0.499008
20	076087	0.077978
30	009239	0.026630
40	028261	0.015136
50	641304	0.568359
60	217391	0.335087
70	157609	0.181323
80	695652	0.284130
90	039348	0.039365

```
In[: ax=sns.distplot(dfr['Actual'],color='r',label='ActualPrice',hist=False)
sns.distplot(dfr['Predicted'],color='g',label='PredictedPrice',ax=ax,hi
```

```
Out[:<Axes: xlabel='Predicted', ylabel='Density'>
```

The red shows the distribution count for actual values and the green shows the distribution count for predicted values. The predicted value line tries to follow the actual value line as closely as possible. The closer the two lines are, the better the model is at predicting the house prices.

In[]:

```
print("R2Score:", r2_score(y_test, r_pred))
print("MeanSquaredError:", mean_squared_error(y_test, r_pred))
print("MeanAbsoluteError:", mean_absolute_error(y_test, r_pred))
print("RootMeanSquaredError:", np.sqrt(mean_squared_error(y_test, r_pred)))
```

```
R2Score:0.8498051402966182
Mean Squared Error:0.005513825572496586
MeanAbsoluteError:0.04537652809312131
RootMeanSquaredError:0.07425513835753446
```

Conclusion

From the exploratory data analysis, we can conclude that the following features are important for predicting the price of a house in Delhi:

1. **Area**
2. **BHK**
3. **Locality**

From the EDA it was also found that, most of the houses in Delhi are small having area between 100 to 200 sq. yard having 2-3 BHK. The price of the houses in localities such as Punjabi Bagh, Lajpat Nagar and Vasant Kunj are high as compared to other localities, which means these are posh areas of Delhi. Most of the people prefer a new builder floor property despite the apartments cost the same because people want to design their

house according to their own needs and requirements and want more privacy and independency.

Coming to the machine learning models, I have used regression models - Decision Tree Regressor and Random Forest Regressor. The Random Forest regressor performed better than the Decision Tree Regressor with an accuracy of 84.98%